

SLAM 3D :

1/ Approche par RTAB-Map :

Le robot a un **RS-Helios** (LiDAR 3D) et une **caméra RealSense D435** — RTAB-Map est conçu exactement pour cette combinaison. Il génère une carte 3D ET une projection 2D utilisable directement par Nav2.

La caméra de profondeur et la caméra RGB non pas la dimension ni le même angle de vue. Il faut donc modifier ces paramètres pour qu'ils concordent.

Dans `~/roskit_ws/src/gr_bringup/launch/component_realsense.launch.py` ajouter :

```
launch_arguments={
    'initial_reset': "True",
    'depth_module.depth_profile': "640,480,30",
    'rgb_camera.profile': "640,480,30",
    'align_depth.enable': "true",
}.items()
```

Le `align_depth.enable` va créer un nouveau topic qui fait en sorte que les paramètres intrinsecs des 2 caméras soit similaire.

Installons les packets manquant :

```
cd ~/roskit_ws
```

```
git clone https://github.com/introlab/rtabmap.git src/rtabmap
```

```
git clone --branch ros2 https://github.com/introlab/rtabmap_ros.git src/rtabmap_ros
```

```
rosdep update && rosdep install --from-paths src --ignore-src -r -y
```

```
colcon build --symlink-install \ --cmake-args -DCMAKE_BUILD_TYPE=Release \ --parallel-workers 1 \ --packages-up-to rtabmap_ros
```

pour les packages trop lourd qui cause l'erreur fatal error c++, les exécuter avec :

```
colcon build --symlink-install --packages-select nom_du_package --parallel-workers 1
```

ça permet de ne pas saturer la ram et de le télécharger, c'est plus lent mais ça fonctionne. Une fois fait refaire « cm » il repartira du cache.

Les topics dans le launch file ne sont pas bons, il faut les renommer. Faites CTRL+F pour chercher « related topics » et « scan_cloud » pour changer :

Avant :

```
# RGB-D related topics
DeclareLaunchArgument('rgb_topic',          default_value='/camera/rgb/image_rect_color',    description=''),
DeclareLaunchArgument('depth_topic',        default_value='/camera/depth_registered/image_raw',  description=''),
DeclareLaunchArgument('camera_info_topic',  default_value='/camera/rgb/camera_info',             description=''),

477
478 # LiDAR
479 DeclareLaunchArgument('subscribe_scan',    default_value='false',    description=''),
480 DeclareLaunchArgument('scan_topic',        default_value='/scan',    description=''),
481 DeclareLaunchArgument('subscribe_scan_cloud', default_value='false',    description=''),
482 DeclareLaunchArgument('scan_cloud_topic',  default_value='/scan_cloud', description=''),
483 DeclareLaunchArgument('scan_normal_k',     default_value='0',        description=''),

499 # imu
500 DeclareLaunchArgument('imu_topic',         default_value='/imu/data',
501 DeclareLaunchArgument('wait_imu_to_init', default_value='false',
502 DeclareLaunchArgument('always_check_imu_tf', default_value='true',
```

Après :

```
453
454 # RGB-D related topics
455 DeclareLaunchArgument('rgb_topic',          default_value='/camera/camera/color/image_raw',    description=''),
456 DeclareLaunchArgument('depth_topic',        default_value='/camera/camera/aligned_depth_to_color/image_raw', description=''),
457 DeclareLaunchArgument('camera_info_topic',  default_value='/camera/camera/color/camera_info',  description=''),

477
478 # LiDAR
479 DeclareLaunchArgument('subscribe_scan',    default_value='false',    description=''),
480 DeclareLaunchArgument('scan_topic',        default_value='/scan',    description=''),
481 DeclareLaunchArgument('subscribe_scan_cloud', default_value='true',    description=''),
482 DeclareLaunchArgument('scan_cloud_topic',  default_value='/rslidar_points', description=''),
483 DeclareLaunchArgument('scan_normal_k',     default_value='0',        description=''),
484

499 # imu
500 DeclareLaunchArgument('imu_topic',         default_value='/imu/data_raw',
501 DeclareLaunchArgument('wait_imu_to_init', default_value='false',
502 DeclareLaunchArgument('always_check_imu_tf', default_value='true',
```

Vérifier que :

```
451 DeclareLaunchArgument('approx_sync', default_value='true',
452 DeclareLaunchArgument('approx_sync_max_interval', default_value='0.02',
```

```

415
416     DeclareLaunchArgument('localization', default_value='false'
417     DeclareLaunchArgument('rtabmap_viz', default_value='false'
418     DeclareLaunchArgument('rviz', default_value='true',

```

Désactive l'odométrie visuelle de RTAB-Map pour n'utiliser que celle du robot

```

485     # Odometry
486     DeclareLaunchArgument('visual_odometry', default_value='false',
436     DeclareLaunchArgument('topic_queue_size', default_value='50',
437     DeclareLaunchArgument('queue_size', default_value='50',

```

```

451     ireLaunchArgument('approx_sync', default_value='true', d
452     ireLaunchArgument('approx_sync_max_interval', default_value='0.02',

```

Une fois tout ça lancer cette commande :

ros2 launch rtabmap_launch rtabmap.launch.py args:="-d"

En cas de problème:

rm ~/.ros/rtabmap.db qui supprime le cache.

Fusion RGB-D Lidar grace à Rtabmap.

```

478     # LiDAR
479     DeclareLaunchArgument('subscribe_scan', default_value='false', des
480     DeclareLaunchArgument('scan_topic', default_value='/scan', des
481     DeclareLaunchArgument('subscribe_scan_cloud', default_value='true', desc
482     DeclareLaunchArgument('scan_cloud_topic', default_value='/rslidar_points',
483     DeclareLaunchArgument('scan_normal_k', default_value='0', des

```

On va utiliser le scan_cloud, car nous avons la chance d'avoir un Lidar 3D donc nous allons l'utiliser.

Relancer avec :

ros2 launch rtabmap_launch rtabmap.launch.py \ args:="-d"

```

485     # Odometry
486     DeclareLaunchArgument('visual_odometry', default_value='false',
487     DeclareLaunchArgument('icp_odometry', default_value='true',
488     DeclareLaunchArgument('odom_topic', default_value='/odom',

```

On active l'icp_odometry. Il est utilisé pour le Scan Matching (ICP - Iterative Closest Point). C'est beaucoup plus précis que la caméra pour aligner les poses du robot au millimètre près.

Modifier le nav2_params.yaml :

```
196     map_subscribe_transient_local: true
197     map topic: /rtabmap/map
```

ros2 launch nav2_bringup navigation_launch.py

Comme dans le TP précédent, il n'y a plus qu'à sectionner 2D Goal pose